



Stack, Queue & Priority Queue

CodeIIEST Summer Bootcamp 2026

Week 04 · Prerequisites

Contents

1	Introduction to Container Adaptors	2
2	Visual Intuition	2
3	Stack	2
3.1	Basic Implementation	2
3.2	Classic Problem: Valid Parentheses	3
4	Queue	4
4.1	Basic Implementation	4
5	Deque (Double Ended Queue)	5
6	Priority Queue	5
6.1	Max-Heap (Default)	6
6.2	Min-Heap (Reversing the Priority)	6
7	When to Use Which Container	7
8	Time Complexity Reference	7
9	Recommended Resources	8

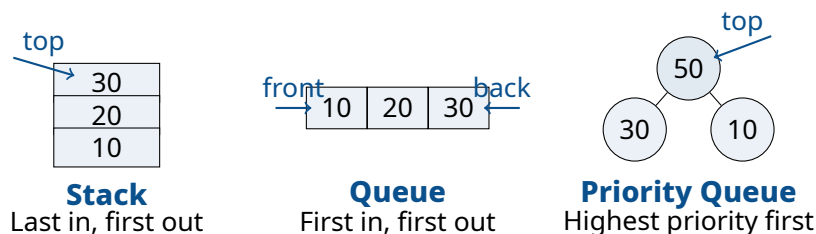
1. Introduction to Container Adaptors

In C++ STL, **Stacks**, **Queues**, and **Priority Queues** are known as *Container Adaptors*. They are not entirely new data structures; instead, they "adapt" (restrict) existing sequence containers (like vector or deque) to provide specific, restricted interfaces.

Crucial Difference from Vectors/Sets:

- You **cannot** iterate through them. There are no `begin()` or `end()` iterators.
- You can only access, add, or remove elements at specific, strictly defined positions (e.g., the top or the front).

2. Visual Intuition



3. Stack

Concept

A Stack is a **LIFO** (Last-In, First-Out) data structure. Think of it like a stack of plates in a cafeteria: the last plate you put on the top of the stack is the very first plate you take off.

Primary Operations (All $O(1)$ Time Complexity):

- `push(x)`: Inserts element x at the top.
- `pop()`: Removes the top element. (Does *not* return it).
- `top()`: Returns the value of the top element.
- `empty()`: Returns `true` if the stack is empty, `false` otherwise.

3.1 Basic Implementation

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     stack<int> st;
```

```
6
7     st.push(10);
8     st.push(20);
9     st.push(30); // Stack from top to bottom: 30, 20, 10
10
11     cout << "Top element is: " << st.top() << "\n"; // Output: 30
12
13     st.pop(); // Removes 30. Stack is now: 20, 10
14
15     cout << "New top is: " << st.top() << "\n";      // Output: 20
16     cout << "Stack size: " << st.size() << "\n";    // Output: 2
17
18     return 0;
19 }
```

CP Warning - Segmentation Fault: Calling `st.top()` or `st.pop()` on an empty stack will cause your program to crash (Runtime Error). Always check `!st.empty()` before accessing the top!

Common Mistakes

- Calling `top()` or `pop()` before checking `empty()`.
- Expecting `pop()` to return the removed value.
- Trying to iterate over a stack directly using `begin()` or `end()`.

3.2 Classic Problem: Valid Parentheses

Stacks are universally used to track matching pairs and histories. A classic problem is checking if a string of brackets is balanced.

```
1 bool isValid(string s) {
2     stack<char> st;
3     for (char c : s) {
4         if (c == '(' || c == '{' || c == '[') {
5             st.push(c); // Push opening brackets
6         } else {
7             if (st.empty()) return false; // Extra closing bracket
8
9             char top = st.top();
10            if ((c == ')' && top == '(') ||
11                (c == '}' && top == '{') ||
12                (c == ']' && top == '[')) {
13                st.pop(); // Match found, remove opening bracket
14            } else {
15                return false; // Mismatch
16            }
17        }
18    }
19    return st.empty(); // True if no unclosed brackets remain
20 }
```

Practice Checkpoints

- **Easy:** Check whether a bracket sequence is balanced.
- **Medium:** Find the next greater element for every array position.
- **Hard:** Compute the largest rectangle in a histogram.

4. Queue

Concept

A Queue is a **FIFO** (First-In, First-Out) data structure. Think of a line of people waiting to buy movie tickets. The first person to join the line is the first person to be served.

Primary Operations (All $O(1)$ Time Complexity):

- `push(x)`: Inserts element x at the **back** of the queue.
- `pop()`: Removes the element at the **front** of the queue.
- `front()`: Returns the value of the front element.
- `back()`: Returns the value of the last element.

4.1 Basic Implementation

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     queue<int> q;
6     q.push(10);
7     q.push(20);
8     q.push(30); // Queue from front to back: 10, 20, 30
9
10    cout << "Front element: " << q.front() << "\n"; // Output: 10
11    cout << "Back element: " << q.back() << "\n";    // Output: 30
12
13    q.pop(); // Removes 10. Queue is now: 20, 30
14    cout << "New front: " << q.front() << "\n";      // Output: 20
15
16    return 0;
17 }
```

Note: Queues are heavily used in Breadth-First Search (BFS) for graphs and trees, which we will cover in later weeks.

Common Mistakes

- Confusing `front()` with `back()` while processing elements.
- Forgetting to check `empty()` before `front()`, `back()`, or `pop()`.
- Using a queue when random access or iteration is required.

5. Deque (Double Ended Queue)

Concept

A Deque (pronounced "deck") bridges the gap between a vector, a stack, and a queue. It allows fast insertion and deletion at **both the front and the back**.

Unlike Stacks and Queues, a Deque **is** a sequence container, meaning you *can* use iterators and the `[]` operator with it.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 int main() {
4     deque<int> dq;
5
6     dq.push_back(10); // {10}
7     dq.push_front(20); // {20, 10}
8     dq.push_back(30); // {20, 10, 30}
9
10    dq.pop_front(); // Removes 20 -> {10, 30}
11    dq.pop_back(); // Removes 30 -> {10}
12    return 0;
13 }
```

Common Mistakes

- Treating a deque like a sorted container; insertion order is preserved.
- Forgetting that `push_front()` and `push_back()` change opposite ends.
- Using a deque when a simple stack or queue interface would be clearer.

Practice Checkpoints

- **Easy:** Simulate browser back and forward navigation.
- **Medium:** Implement a sliding window maximum.

6. Priority Queue

Concept

A Priority Queue is a queue where elements are served based on their **priority** rather than their insertion order. By default in C++, a `priority_queue` acts as a **Max-Heap**, meaning the largest element is always at the top.

Under the hood, Priority Queues are implemented using Heaps (complete binary trees). Insertion and extraction take $O(\log N)$ time, while viewing the top takes $O(1)$ time.

6.1 Max-Heap (Default)

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     // Max-Heap by default
6     priority_queue<int> pq;
7     pq.push(10);
8     pq.push(50);
9     pq.push(30);
10
11     cout << "Top element: " << pq.top() << "\n"; // Output: 50
12     pq.pop(); // Removes 50. New top will be 30.
13
14     return 0;
15 }
```

6.2 Min-Heap (Reversing the Priority)

To make the *smallest* element stay at the top (Min-Heap), the syntax is slightly longer. We must pass the underlying container type (vector) and a comparator (greater).

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 int main() {
4     // Min-Heap syntax
5     priority_queue<int, vector<int>, greater<int>> min_pq;
6     min_pq.push(10);
7     min_pq.push(50);
8     min_pq.push(5);
9
10     cout << "Smallest element: " << min_pq.top() << "\n"; // Output: 5
11     return 0;
12 }
```

CP Trick: If you forget the Min-Heap syntax during a contest, you can just use a default Max-Heap but push the **negative** versions of your numbers (`pq.push(-x)`). When you pop them, negate them back. The largest negative number corresponds to the smallest positive number!

Common Mistakes

- Forgetting that the default `priority_queue` is a max-heap.
- Expecting equal-priority elements to preserve insertion order.
- Updating priorities in place; usually you push a new state and ignore stale ones.

Practice Checkpoints

- **Easy:** Print numbers in descending priority order.
- **Medium:** Merge K sorted arrays or lists.

7. When to Use Which Container

Container	Use when...	Typical Pattern
Stack	You need to undo the latest action or match nested pairs.	Parentheses, monotonic stack, DFS history.
Queue	You must process items in arrival order.	BFS, simulations, level-order traversal.
Deque	You need fast insertion or removal at both ends.	Sliding window, 0-1 BFS, double-ended simulation.
Priority Queue	The next item depends on best priority, not arrival time.	Dijkstra, greedy scheduling, top K elements.

8. Time Complexity Reference

Below is a summary of the Big-O complexities for the adaptors.

Data Structure	Insertion	Deletion	Access Top/Front	Search
Stack	$O(1)$	$O(1)$	$O(1)$	$O(N)$
Queue	$O(1)$	$O(1)$	$O(1)$	$O(N)$
Deque	$O(1)$ (Front/Back)	$O(1)$ (Front/Back)	$O(1)$	$O(N)$
Priority Queue	$O(\log N)$	$O(\log N)$	$O(1)$	N/A

9. Recommended Resources

Stacks, queues, and heaps are foundational for more advanced topics like Graph Theory and greedy algorithms. Practice the basics with these resources:

- **USACO Guide:**

- [Gold: Stacks](#)
- [Silver: Priority Queues](#)

- **GeeksforGeeks (GFG):**

- [Stack Data Structure Overview](#)
- [Priority Queue in C++ STL](#)

- **YouTube Channels:**

- [Take U Forward / Striver](#) (Excellent crash course on Stack & Queue Implementation).
- [Aditya Verma: Stack Playlist](#) (The absolute best playlist for identifying and solving standard Stack patterns like Next Greater Element).